

05_Neural_Network_Bjoern

April 10, 2026

1 Neuronales Nezt

Disclaimer: Aufgrund der langen Zeit die das gesamte Notebook zum asuführen benötigt, besteht der Code aus einzelnen Schnipseln und kann nicht als ganzes Wahrgenommen werden. Vielmehr nutze ich jeden Tag nur einzelne Zellen. Ich habe die Funktionsweise dieses Notebooks nicht dokumentiert.

Importieren der Module und Einfügen der Transformatierten Daten:

```
[ ]: import sys
      sys.path.insert(0, '..')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.linear_model import LinearRegression

from utils.data import load_and_clean_data, get_train_test_split
from utils.evaluation import evaluate_predictions, add_result,
    ↪ evaluate_test_predictions
from utils.plotting import plot_features_vs_target, plot_predicted_vs_actual,
    ↪ plot_residuals, save_fig

# jetzt noch eigene

from keras import regularizers

plt.rcParams['figure.dpi'] = 100
%matplotlib inline

print(f"TensorFlow Version: {tf.__version__}")
```

```

print(f"Numpy Version: {np.__version__}")
print(f"Pandas Version: {pd.__version__}")
print(f"Matplotlib Version: {plt.matplotlib.__version__}")
print(f"python Version: {sys.version}")

```

```

[ ]: import os, sys

project_root = os.path.abspath("..") # von notebooks/ eine Ebene hoch
if project_root not in sys.path:
    sys.path.insert(0, project_root)

print(project_root)

from utils.data import load_and_clean_data, get_train_test_split
df = load_and_clean_data()
X_train, X_test, y_train, y_test, feature_names = get_train_test_split(df)

# Validierungssplit aus Trainingsdaten
val_split = int(0.8 * len(X_train))
X_val, y_val = X_train[val_split:], y_train[val_split:]
X_train, y_train = X_train[:val_split], y_train[:val_split]

print(f"Training:    {X_train.shape}")
print(f"Validation: {X_val.shape}")
print(f"Test:         {X_test.shape}")
print(f"Features:     {feature_names}")

X_train_standard, X_test_standard, y_train_standard, y_test_standard, scaler,
↪ feature_names = X_train, X_test, y_train, y_test, scaler, feature_names =
↪ get_train_test_split(df, scaler='standard')

val_split = int(0.8 * len(X_train_standard))
X_val_standard, y_val_standard = X_train_standard[val_split:],
↪ y_train_standard[val_split:]
X_train_standard, y_train_standard = X_train_standard[:val_split],
↪ y_train_standard[:val_split]

print(f"Training:    {X_train_standard.shape}")
print(f"Validation: {X_val_standard.shape}")
print(f"Test:         {X_test_standard.shape}")
print(f"Features:     {feature_names}")

X_train_min0_max1, X_test_min0_max1, y_train_min0_max1, y_test_min0_max1,
↪ scaler, feature_names = get_train_test_split(df, scaler='minmax')

```

```

val_split = int(0.8 * len(X_train_min0_max1))
X_val_min0_max1, y_val_min0_max1 = X_train_min0_max1[val_split:],
    ↪ y_train_min0_max1[val_split:]
X_train_min0_max1, y_train_min0_max1 = X_train_min0_max1[:val_split],
    ↪ y_train_min0_max1[:val_split]

print(f"Training:    {X_train_min0_max1.shape}")
print(f"Validation:  {X_val_min0_max1.shape}")
print(f"Test:        {X_test_min0_max1.shape}")
print(f"Features:     {feature_names}")

```

1.1 Erste Versuche

Aus Felix H.s Ergebnissen ist zu lernen, dass nur 4 Argumente einen Informationsgewinn haben. Diese sind: ["MedInc", "AveOccup", "Latitude", "Longitude"]. Es werden die standardisierten Daten genommen.

```

[ ]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"] # aus
    ↪ rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in
    ↪ important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

```

Jetzt erstelle ich mein erstes Netzwerk mit den 4 Argumenten. Es stellte sich zuvor heraus, dass L2 Regularisierung sinnvoll ist

```

[ ]: from utils.evaluation import evaluate_model
model_1 = keras.Sequential([
    layers.Input(shape=(X_train_important.shape[1],)),
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(16, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(1)
])
# kompilieren

```

```

model_1.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
↳loss='mse', metrics=['mae'])

# fitten

model_1.fit(
    X_train_important, y_train_important,
    validation_data=(X_val_important, y_val_important),
    epochs=100,
    batch_size=32,
    callbacks=[
        keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)
    ]
)

# trainieren
result_1 = evaluate_model(model_1, X_train_important, X_test_important,
↳y_train_important, y_test_important, model_name="NN_important_features")

```

Das erste hier gebaute Modell ist schonmal ein guter Start. Es ist kleiner als die besten Modelle der anderen, sollte aber auch erstmal ein Test sein.

Als nächster Schritt wird das Netz größer gebaut mit einem Layer mehr.

```

[ ]: from utils.evaluation import evaluate_model
model_1 = keras.Sequential([
    layers.Input(shape=(X_train_important.shape[1],)),
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.
↳001)),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.
↳001)),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.
↳001)),
    layers.Dropout(0.2),
    layers.Dense(16, activation='relu', kernel_regularizer=regularizers.l2(0.
↳001)),
    layers.Dropout(0.2),
    layers.Dense(1)
])

# kompilieren
model_1.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
↳loss='mse', metrics=['mae'])

# fitten

```

```

model_1.fit(
    X_train_important, y_train_important,
    validation_data=(X_val_important, y_val_important),
    epochs=100,
    batch_size=32,
    callbacks=[
        keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)
    ]
)

# trainieren
result_1 = evaluate_model(model_1, X_train_important, X_test_important,
    ↪ y_train_important, y_test_important, model_name="NN_important_features")

```

Der zusätzliche Layer hat das Ergebnis nur maginal verbessert. Bei beiden Versuchen bricht das Netz schon früh ab, deswegen wird hier mal der patience Wert erhöht.

```

[ ]: from utils.evaluation import evaluate_model
model_1 = keras.Sequential([
    layers.Input(shape=(X_train_important.shape[1],)),
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(16, activation='relu', kernel_regularizer=regularizers.l2(0.
    ↪ 001)),
    layers.Dropout(0.2),
    layers.Dense(1)
])
# kompilieren
model_1.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
    ↪ loss='mse', metrics=['mae'])

# fitten
model_1.fit(
    X_train_important, y_train_important,
    validation_data=(X_val_important, y_val_important),
    epochs=100,
    batch_size=32,
    callbacks=[

```

```

        keras.callbacks.EarlyStopping(patience=15, restore_best_weights=True)
    ]
)

# trainieren
result_1 = evaluate_model(model_1, X_train_important, X_test_important,
    ↪ y_train_important, y_test_important, model_name="NN_important_features")

```

Das hat nicht viel gebracht

1.2 Nun geht es an die richtige Arbeit

Ich möchte Random-Search zu ausprobieren. Dafür lohnt es sich die Größenordnungen der Hyperparameter einzugrenzen.

- der Neuronen in den Schichten: 32, 64, 128
- Anzahl der Schichten: 2, 3
- Lernrate: Logarithmisch zwischen 0.0001 und 0.01
- Regularisierung: L2 mit einem Faktor von 0.000 01 bis 0.001
- Dropout-Rate: 0.1 bis 0.3
- Batch-Größe: 16, 32, 64
- Anzahl der Epochen: 50 bis 200
- Aktivierungsfunktion: macht keinen großen Unterschied, ReLu ist günstig

Ich befürchte eine große Rechenzeit, weil ich mehr als nur eine Handvoll Modelle testen möchte. Daher Frage an ChatGPT: "ich gehe mal davon aus, dass das ganze viel rechenzeit benötigt. kann man das optimieren? ich habe da z.b. mal was von parallelisierung gehört" - Antwort: zum Sparen von Rechenzeit: - Early Stopping mit einer Geduld von 5 Epochen - CPU Parallelisierung - weniger Epochen max 100

```

[ ]: import numpy as np
import pandas as pd
import random
from joblib import Parallel, delayed
from tensorflow import keras
from tensorflow.keras import layers, regularizers

# -----
# Modell bauen (abhängig von HPs)
# -----
def build_model(input_dim, params):
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))

    units = params["units"]
    depth = params["depth"]
    dropout_rate = params["dropout"]

```

```

12_reg = params["l2"]

for i in range(depth):
    model.add(layers.Dense(
        units // (2**i),
        activation="relu",
        kernel_regularizer=regularizers.l2(12_reg)
    ))
    model.add(layers.Dropout(dropout_rate))

model.add(layers.Dense(1))

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=params["lr"]),
    loss="mse",
    metrics=["mae"]
)

return model

# -----
# Zufällige Hyperparameter ziehen
# -----
def sample_params():
    return {
        "units": random.choice([32, 64, 128]),
        "depth": random.choice([2, 3]),
        "lr": 10 ** np.random.uniform(-4, -2),
        "l2": 10 ** np.random.uniform(-5, -3),
        "dropout": np.random.uniform(0.1, 0.3),
        "batch_size": random.choice([16, 32, 64]),
        "epochs": random.randint(50, 100)
    }

# -----
# Ein einzelner Run
# -----
def train_one_run(run_id, X_train, y_train, X_val, y_val, X_test, y_test):

    params = sample_params()
    model = build_model(X_train.shape[1], params)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),

```

```

        epochs=params["epochs"],
        batch_size=params["batch_size"],
        verbose=0,
        callbacks=[
            keras.callbacks.EarlyStopping(
                patience=7,
                restore_best_weights=True
            )
        ]
    )

    # Evaluation
    y_test_pred = model.predict(X_test, verbose=0)
    y_val_pred = model.predict(X_val, verbose=0)

    from sklearn.metrics import r2_score

    result = {
        "run_id": run_id,
        "R2 Val": r2_score(y_val, y_val_pred),
        "R2 Test": r2_score(y_test, y_test_pred),
        **params
    }

    return result, model

# -----
# Random Search (parallel)
# -----
def random_search(
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    n_runs=20,
    n_jobs=1 # >1 = parallel
):
    results = []
    models = []

    runs = Parallel(n_jobs=n_jobs)(
        delayed(train_one_run)(
            i, X_train, y_train, X_val, y_val, X_test, y_test
        )
        for i in range(n_runs)
    )

```



```

for result, model in runs:
    results.append(result)
    models.append(model)

df = pd.DataFrame(results)

# Bestes Modell anhand Validation auswählen!
best_idx = df["R2 Val"].idxmax()

best_model = models[best_idx]
best_params = results[best_idx]

return best_model, best_params, df

```

```

[ ]: best_model, best_params, results_df = random_search(
    X_train_important, y_train_important,
    X_val_important, y_val_important,
    X_test_important, y_test_important,
    n_runs=20, ##### HIER #####
    n_jobs=4 # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R2 Val", ascending=False).head()

```

Die Ergebnisse zeigen kaum Overfitting-Tendenzen (Differenz R^2 des besten Netzes 0.014) und liegen bei einem R^2 von 0.73. Das ist für ökonomische Zwecke ein sehr guter Score(text) Die beiden besten Ergebnisse zeigen ähnliche Ergebnisse und bilden eine sich abgestzte Spitze. Es wird im Folgenden der Code neu ausgeführt, um mehr Ergebnisse zu bekommen. Das jetzige Ergebnis wird unter einer neuen Variable gespeichert. Ziel ist es Tendenzen zu sehen, für bestimmte Kombinationen

mehrere Durchläufe zum aufklappen

```

[ ]: Durchlauf_1 = results_df.sort_values("R2 Test", ascending=False).head(2)
    Durchlauf_1

[ ]: Durchlauf_2 = results_df.sort_values("R2 Test", ascending=False).head(10)

[ ]: Durchlauf_3 = results_df.sort_values("R2 Test", ascending=False).head(9)
    Durchlauf_3

[ ]: Durchlauf_4 = results_df.sort_values("R2 Test", ascending=False).head(10)
    Durchlauf_4

```

Die Ergebnisse werden nun zusammengetragen.

```
[ ]: Ergebnis_bis_dato = pd.concat([Durchlauf_1, Durchlauf_2,Durchlauf_3,
    ↳Durchlauf_4], axis=0) # Verbindet DataFrames vertikal (übereinander,
    ↳Standard)
Ergebnis_bis_dato.drop("run_id", axis=1, inplace=True)
Ergebnis_bis_dato.reset_index(drop=True, inplace=True)
Ergebnis_bis_dato
```

Der Datensatz wird erweitert

```
[ ]: import pandas as pd
gestern = pd.read_csv(r"C:
    ↳\KI_2026\KI_308\ki-308\results\random_search_best_results.csv")

[ ]: neue_results = results_df.sort_values("R2 Test", ascending=False)
neue_results.drop("run_id", axis=1, inplace=True)
neue_results.reset_index(drop=True, inplace=True)
Ergebnisse_bis_dato = pd.concat([gestern, neue_results.head(20)], axis=0)
Ergebnisse_bis_dato.sort_values("R2 Test", ascending=False, inplace=True)
Ergebnisse_bis_dato.reset_index(drop=True, inplace=True)
Ergebnisse_bis_dato
```

Zur Einordnung wird ein RandomForestGenerator genutzt. Der RFG soll die Zusammenhänge der Hyperparameter aufdecken und somit das Verständnis von neuronalen Netzen bei dem California Housing Datensatz verbessern.

Anhand der PDP können die Hyperparameter weiter eingeschränkt werden. Ziel ist ein Iteratives Verfahren, mit dem man die beste Kombination von Hyperparametern erfassen kann.

Das ganze ist ein Meta-/Übermodell

```
[ ]: df = Ergebnisse_bis_dato.copy()

# Log-Transformation der Hyperparameter für bessere Visualisierung
df["lr_log"] = np.log10(df["lr"])
df["l2_log"] = np.log10(df["l2"])
# Epochen werden hier ausgelassen
X = df[["units", "depth", "lr_log", "l2_log", "dropout","batch_size"]]
Y = df["R2 Test"]

from sklearn.ensemble import RandomForestRegressor

meta_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

meta_model.fit(X, Y)
```

```
[ ]: from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

features_to_plot = ["units", "depth", "batch_size", "dropout", "lr_log", "l2_log"]

fig, ax = plt.subplots(figsize=(12, 8))

PartialDependenceDisplay.from_estimator(
    meta_model,
    X,
    features=features_to_plot,
    grid_resolution=20,
    ax=ax
)
# Plot speichern
plt.savefig("pdp.png", dpi=300)
plt.tight_layout()
plt.show()
```

Spannend ist auch, wie sehr die Hyperparameter zum Gesamtscore beitragen.

```
[ ]: import pandas as pd

importance = pd.Series(meta_model.feature_importances_, index=["units", "depth", "lr_log", "l2_log", "dropout", "batch_size"])
print(importance.sort_values(ascending=False), type(importance.sort_values(ascending=False)))
```

```
[ ]: # Speichern der ergebnisse

import joblib

# Data speichern
Ergebnis_bis_dato.to_csv("random_search_best_results.csv", index=False)

# Plot speichern
plt.savefig("pdp.png", dpi=300)
```

```
[ ]: importance.sort_values(ascending=False).plot(kind="bar")
```

1.3 zweite (und dritte) Iteration mit angepassten Parametern

Die Schritte von der ersten Iteration werden übernommen

```
[ ]: import numpy as np
import pandas as pd
import random
from joblib import Parallel, delayed
from tensorflow import keras
from tensorflow.keras import layers, regularizers

# -----
# Modell bauen (abhängig von HPs)
# -----
def build_model(input_dim, params):
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))

    units = params["units"]
    depth = params["depth"]
    dropout_rate = params["dropout"]
    l2_reg = params["l2"]

    for i in range(depth):
        model.add(layers.Dense(
            units // (2**i),
            activation="relu",
            kernel_regularizer=regularizers.l2(l2_reg)
        ))
        model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=params["lr"]),
        loss="mse",
        metrics=["mae"]
    )

    return model

# -----
# Zufällige Hyperparameter ziehen
# -----
def sample_params():
    return {
        "units": random.choices([32, 64, 128], weights=[1,2,4])[0],
        "depth": random.choice([2, 3]),
        "lr": 10 ** np.random.uniform(-6, -4),
    }
```

```

        "l2": 10 ** np.random.uniform(-5, -3),
        "dropout": np.random.uniform(0.115, 0.16),
        "batch_size": random.choice([16, 32]),
        "epochs": 200
    }

# -----
# Ein einzelner Run
# -----
def train_one_run(run_id, X_train, y_train, X_val, y_val, X_test, y_test):

    params = sample_params()
    model = build_model(X_train.shape[1], params)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=params["epochs"],
        batch_size=params["batch_size"],
        verbose=0,
        callbacks=[
            keras.callbacks.EarlyStopping(
                patience=7,
                restore_best_weights=True
            )
        ]
    )

    # Evaluation
    y_test_pred = model.predict(X_test, verbose=0)
    y_val_pred = model.predict(X_val, verbose=0)

    from sklearn.metrics import r2_score

    result = {
        "run_id": run_id,
        "R2 Val": r2_score(y_val, y_val_pred),
        "R2 Test": r2_score(y_test, y_test_pred),
        **params
    }

    return result, model

# -----
# Random Search (parallel)

```

```

# -----
def random_search(
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    n_runs=20,
    n_jobs=1  # >1 = parallel
):

    results = []
    models = []

    runs = Parallel(n_jobs=n_jobs)(
        delayed(train_one_run)(
            i, X_train, y_train, X_val, y_val, X_test, y_test
        )
        for i in range(n_runs)
    )

    for result, model in runs:
        results.append(result)
        models.append(model)

    df = pd.DataFrame(results)

    # Bestes Modell anhand Validation auswählen!
    best_idx = df["R2 Val"].idxmax()

    best_model = models[best_idx]
    best_params = results[best_idx]

    return best_model, best_params, df

```

```

[ ]: best_model, best_params, results_df = random_search(
    X_train_important, y_train_important,
    X_val_important, y_val_important,
    X_test_important, y_test_important,
    n_runs=10 , ##### HIER #####
    n_jobs=4  # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R2 Val", ascending=False).head()

```

```
[ ]: nummer_3 = results_df.sort_values("R2 Test", ascending=False).head(1)
nummer_3
```

```
[ ]: nummer_2 = results_df.sort_values("R2 Test", ascending=False).head(20)
nummer_2
```

```
[ ]: nummer_1 = results_df.sort_values("R2 Test", ascending=False).head(20)
nummer_1
```

```
[ ]: df = nummer_1.copy()

# Log-Transformation der Hyperparameter für bessere Visualisierung
df["lr_log"] = np.log10(df["lr"])
df["l2_log"] = np.log10(df["l2"])
# Epochen werden hier ausgelassen
X = df[["units", "depth", "lr_log", "l2_log", "dropout", "batch_size"]]
Y = df["R2 Test"]
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
meta_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)
```

```
meta_model.fit(X, Y)
```

```
[ ]: from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

features_to_plot = ["units", "depth", "batch_size", "dropout", "lr_log", ↵
↵ "l2_log"]

fig, ax = plt.subplots(figsize=(12, 8))

PartialDependenceDisplay.from_estimator(
    meta_model,
    X,
    features=features_to_plot,
    grid_resolution=20,
    ax=ax
)

# Plot speichern
plt.savefig("pdp.png", dpi=300)
plt.tight_layout()
plt.show()
```

```
[ ]: importance.sort_values(ascending=False).plot(kind="bar")
```

```
[ ]: Iter_3 = pd.concat([nummer_1, nummer_2, nummer_3])
Iter_3.sort_values("R² Test", ascending=False, inplace=True)
Iter_3.reset_index(drop=True, inplace=True)
Iter_3.drop("run_id", axis=1, inplace=True)
Iter_3
```

```
[ ]: from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

features_to_plot = ["units", "depth", "batch_size", "dropout", "lr_log",
↳ "l2_log"]

fig, ax = plt.subplots(figsize=(12, 8))

PartialDependenceDisplay.from_estimator(
    meta_model,
    X,
    features=features_to_plot,
    grid_resolution=20,
    ax=ax
)
# Plot speichern
# plt.savefig("pdp.png", dpi=300)
plt.tight_layout()
plt.show()
```

```
[ ]: importance = pd.Series(meta_model.feature_importances_, index=["units",
↳ "depth", "lr_log", "l2_log", "dropout", "batch_size"])
importance.sort_values(ascending=False).plot(kind="bar")
```

```
[ ]: Iter_3.to_csv("random_search_best_results_Iter_3.csv", index=False)
```

Stand 19.03. 18.07 Uhr

2 20.03.

```
[ ]: # ich möchte nicht immer den Code von vorher ausführen müssen, daher vermehrte
↳ Importe der gleichen Module
import numpy as np
import pandas as pd
import random
from joblib import Parallel, delayed
from tensorflow import keras
from tensorflow.keras import layers, regularizers
```



```

# -----
# Modell bauen (abhängig von HPs)
# -----
def build_model(input_dim, params):
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))

    units = params["units"]
    depth = params["depth"]
    dropout_rate = params["dropout"]
    l2_reg = params["l2"]

    for i in range(depth):
        model.add(layers.Dense(
            units // (2**i),
            activation="relu",
            kernel_regularizer=regularizers.l2(l2_reg)
        ))
        model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=params["lr"]),
        loss="mse",
        metrics=["mae"]
    )

    return model

# -----
# Zufällige Hyperparameter ziehen
# -----
def sample_params():
    return {
        "units": random.choices([32, 64, 128], weights=[2,3,4])[0],
        "depth": random.choice([2, 3, 4]),
        "lr": 10 ** np.random.uniform(-3.3, -2),
        "l2": 10 ** np.random.uniform(-5, -3),
        "dropout": np.random.uniform(0.115, 0.16),
        "batch_size": random.choice([16, 32]),
        "epochs": 500
    }

```

```

# -----
# Ein einzelner Run
# -----
def train_one_run(run_id, X_train, y_train, X_val, y_val, X_test, y_test):

    params = sample_params()
    model = build_model(X_train.shape[1], params)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=params["epochs"],
        batch_size=params["batch_size"],
        verbose=0,
        callbacks=[
            keras.callbacks.EarlyStopping(
                patience=7,
                restore_best_weights=True
            )
        ]
    )

    # Evaluation
    y_test_pred = model.predict(X_test, verbose=0)
    y_val_pred = model.predict(X_val, verbose=0)

    from sklearn.metrics import r2_score

    result = {
        "run_id": run_id,
        "R2 Val": r2_score(y_val, y_val_pred),
        "R2 Test": r2_score(y_test, y_test_pred),
        **params
    }

    return result, model

# -----
# Random Search (parallel)
# -----
def random_search(
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    n_runs=20,
    n_jobs=1 # >1 = parallel

```

```

):

results = []
models = []

runs = Parallel(n_jobs=n_jobs)(
    delayed(train_one_run)(
        i, X_train, y_train, X_val, y_val, X_test, y_test
    )
    for i in range(n_runs)
)

for result, model in runs:
    results.append(result)
    models.append(model)

df = pd.DataFrame(results)

# Bestes Modell anhand Validation auswählen!
best_idx = df["R2 Val"].idxmax()

best_model = models[best_idx]
best_params = results[best_idx]

return best_model, best_params, df

```

```

[ ]: best_model, best_params, results_df = random_search(
    X_train_important, y_train_important,
    X_val_important, y_val_important,
    X_test_important, y_test_important,
    n_runs=200 , ##### HIER #####
    n_jobs=4   # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R2 Val", ascending=False).head()

```

```

[ ]: results_df.sort_values("R2 Test", ascending=False).head(50)

```

```

[ ]: add_today_1 = results_df.sort_values("R2 Test", ascending=False).head(20)

```

3 Vorbereitungen für Ensembles

ich möchte aus meinen besten Hyperparameterkonfigurationen ein Ensemble bauen.

```
[ ]: # Alles von gestern + die neuen besten 4 von heute
gestern_19 = pd.read_csv("C:
↳\\KI_2026\\KI_308\\ki-308\\results\\random_search_best_results_Iter_3.csv")
neue_results = results_df.sort_values("R² Test", ascending=False)
neue_results.drop("run_id", axis=1, inplace=True)
neue_results.reset_index(drop=True, inplace=True)
Ergebnis_bis_20 = pd.concat([gestern_19, neue_results.head(4)], axis=0)
Ergebnis_bis_20.sort_values("R² Test", ascending=False, inplace=True)
Ergebnis_bis_20.reset_index(drop=True, inplace=True)
Ergebnis_bis_20
```

```
[ ]: concat = pd.concat([Ergebnis_bis_20, add_today_1, results_df.head(50)], axis=0)
concat.sort_values("R² Test", ascending=False, inplace=True)
concat.reset_index(drop=True, inplace=True)
concat.sort_values("R² Test", ascending=False)
concat.drop("run_id", axis=1, inplace=True)
concat.head(50)
```

Suche die besten 10 heraus, die auch möglichst divers sind

```
[ ]: ensemble_df = pd.DataFrame(concat.iloc[[0,1,5,6,9,10,12,16, 18, 20, 21, 22]])
ensemble_df.reset_index(drop=True, inplace=True)
ensemble_df
```

```
[ ]: def sample_params():
    return {
        "units": random.choice([32, 64, 128]),
        "depth": random.choices([2, 3, 4], weights=[2,3,4])[0],
        "lr": 10 ** np.random.uniform(-4, -2),
        "l2": 10 ** np.random.uniform(-5, -3),
        "dropout": np.random.uniform(0.1, 0.2),
        "batch_size": random.choice([16, 32, 64]),
        "epochs": random.choice([200, 300, 400, 500])
    }
```

```
[ ]: best_model, best_params, results_df = random_search(
    X_train_important, y_train_important,
    X_val_important, y_val_important,
    X_test_important, y_test_important,
    n_runs=100 , ##### HIER #####
    n_jobs=4 # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R² Val", ascending=False).head()
```

```
[ ]: ensemble_df = pd.DataFrame(concat.iloc[[0,1,5,6,9,10,12,16, 18, 20, 21, 22]])
ensemble_df.reset_index(drop=True, inplace=True)
ensemble_df = pd.concat([ensemble_df, results_df.iloc[[9,1, 16, 17, 33]]])
ensemble_df.sort_values("R2 Test", ascending=False, inplace=True)
ensemble_df.reset_index(drop=True, inplace=True)
ensemble_df.drop([6,7,8],inplace=True)
```

```
[ ]: ensemble_df.reset_index(drop=True, inplace=True)
ensemble_df
```

```
[ ]: results_df = results_df.sort_values("R2 Test", ascending=False)

results_df.reset_index(drop=True, inplace=True)
results_df.head(50)
```

4 Ensemble

4.0.1 Ensemble mit mean (d.h. nicht gewichtet)

bis jetzt habe ich nur Hyperparameterkombinationen. Die muss ich noch zu Netzen machen und trainieren. Die trainierten Netze kommen dann in eine Liste.

```
[ ]: trained_models = []

for _, row in ensemble_df.iterrows():

    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))

    for i in range(int(row["depth"])):
        model.add(layers.Dense(
            int(row["units"]),
            activation="relu",
            kernel_regularizer=regularizers.l2(float(row["l2"]))
        ))
        model.add(layers.Dropout((float(row["dropout"]))))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=float(row["lr"])),
        loss="mse",
        metrics=["mae"]
    )
    model.fit(
        X_train_important, y_train_important,
        validation_data=(X_val_important, y_val_important),
```

```

        epochs=int(row["epochs"]),
        batch_size=int(row["batch_size"]),
        callbacks=[keras.callbacks.EarlyStopping(patience=10,
↪restore_best_weights=True)],
        verbose=0
    )
    trained_models.append(model)

```

```

[ ]: def ensemble_predict(models, X):
    preds = np.array([
        model.predict(X).flatten()
        for model in models
    ])
    return np.mean(preds, axis=0)

```

```

[ ]: y_pred_test = ensemble_predict(trained_models, X_test_important)
    evaluate_test_predictions(
        y_test_important,
        y_pred_test,
        "Ensemble"
    )

```

Das Ergebnis des Ensembles ist wirklich enttäuschend. Grund könnte sein, dass die Hyperparameter schon jeweils so stark an einander angepasst waren, dass die Diversität am Ende zu klein war. Die Gewichtung stellt wahrscheinlich kein großes Problem dar, da alle verwendeten Netze ähnlich gut waren.

```

[ ]: # nochmal den Dataframe für das Ensemble speichern
    ensemble_df.to_csv("ensemble_models.csv", index=False)

```

Stand 20.03. 17.29 Uhr

5 Features Bearbeiten

Ich erhoffe, dass durch das Verändern der Features, also den gesamten California Housing Datensatz zu verwenden + weitere, ich ein besseres Ergebnis habe, da mehr Daten zum trainieren vorhanden sind.

5.0.1 zuerst mal den ganzen Datensatz auf das Ensemble trainieren.

```

[ ]: ensemble_df = pd.read_csv(r"C:\KI_2026\KI_308\ki-308\results\ensemble_models.
↪csv")

```

```

[ ]: trained_models = []

    for _, row in ensemble_df.iterrows():

```

```

model = keras.Sequential()
model.add(layers.Input(shape=(X_train_standard.shape[1],)))

for i in range(int(row["depth"])):
    model.add(layers.Dense(
        int(row["units"]),
        activation="relu",
        kernel_regularizer=regularizers.l2(float(row["l2"]))
    ))
    model.add(layers.Dropout((float(row["dropout"]))))

model.add(layers.Dense(1))

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=float(row["lr"])),
    loss="mse",
    metrics=["mae"]
)
model.fit(
    X_train_standard, y_train_standard,
    validation_data=(X_val_standard, y_val_standard),
    epochs=int(row["epochs"]),
    batch_size=int(row["batch_size"]),
    callbacks=[keras.callbacks.EarlyStopping(patience=10,
↪restore_best_weights=True)],
    verbose=0
)
trained_models.append(model)

```

```

[ ]: def ensemble_predict(models, X):
    preds = np.array([
        model.predict(X).flatten()
        for model in models
    ])
    return np.mean(preds, axis=0)

```

```

[ ]: y_pred_test = ensemble_predict(trained_models, X_test_standard)
evaluate_test_predictions(
    y_test_standard,
    y_pred_test,
    "Ensemble"
)

```

Das Ergebnis ist nun deutlich besser als vorher.

5.0.2 jetzt Feature Engineering

hinzugefügt werden: - AveRooms / AveOccup, - AveBedrms / AveRooms, - Population / AveOccup, - MedInc / AveOccup

Es sollten die normierten Daten verwendet werden.

```
[ ]: def feature_engineering(df, feature_names):
    df = pd.DataFrame(df, columns=feature_names).copy()

    eps = 1e-8 # Vermeidung von Division durch Null
    df["rooms per household"] = df["AveRooms"] / (df["AveOccup"] + eps)
    df["bedrooms per room"] = df["AveBedrms"] / (df["AveRooms"] + eps)
    df["population per household"] = df["Population"] / (df["AveOccup"] + eps)
    df["income per household"] = df["MedInc"] / (df["AveOccup"] + eps)

    return df
```

```
[ ]: # Feature Engineering auf minmax-skalierten Daten
feature_names = ["MedInc", "HouseAge", "AveRooms", "AveBedrms",
                 "Population", "AveOccup", "Latitude", "Longitude"]

X_train_engineered = feature_engineering(X_train_min0_max1, feature_names)
X_val_engineered    = feature_engineering(X_val_min0_max1, feature_names)
X_test_engineered   = feature_engineering(X_test_min0_max1, feature_names)

print("Train shape:", X_train_engineered.shape)
print("Val shape:",   X_val_engineered.shape)
print("Test shape:",  X_test_engineered.shape)
```

Nun habe ich 12 Features. Mal schauen was passiert:

```
[ ]: trained_models = []

for _, row in ensemble_df.iterrows():

    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_min0_max1.shape[1],)))

    for i in range(int(row["depth"])):
        model.add(layers.Dense(
            int(row["units"]),
            activation="relu",
            kernel_regularizer=regularizers.l2(float(row["l2"])))
        ))
    model.add(layers.Dropout(float(row["dropout"])))
```



```

model.add(layers.Dense(1))

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=float(row["lr"])),
    loss="mse",
    metrics=["mae"]
)
model.fit(
    X_train_min0_max1, y_train_min0_max1,
    validation_data=(X_val_min0_max1, y_val_min0_max1),
    epochs=int(row["epochs"]),
    batch_size=int(row["batch_size"]),
    callbacks=[keras.callbacks.EarlyStopping(patience=10,
↪restore_best_weights=True)],
    verbose=0
)
trained_models.append(model)

```

```

[ ]: def ensemble_predict(models, X):
    preds = np.array([
        model.predict(X).flatten()
        for model in models
    ])
    return np.mean(preds, axis=0)

```

```

[ ]: y_pred_test = ensemble_predict(trained_models, X_test_min0_max1)
evaluate_test_predictions(
    y_test_min0_max1,
    y_pred_test,
    "Ensemble"
)

```

Der Score hat sich durch die zusätzlichen Features tatsächlich verschlechtert. Ich habe das Gefühl, dass die Netzstruktur sehr weit eingeschränkt ist und deswegen den erweiterten Datensatz nicht gut abbilden kann. Deswegen werde ich noch einmal Random Search anwenden. Ziel ist es erneut gute Netze zu finden und dann schlussendlich ein Ensemble zu kreieren aus dem alten Ensemble mit den neuen Netzen.

```

[ ]: import numpy as np
import pandas as pd
import random
from joblib import Parallel, delayed
from tensorflow import keras
from tensorflow.keras import layers, regularizers

```

```

# -----
# Modell bauen (abhängig von HPs)
# -----
def build_model(input_dim, params):
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))

    units = params["units"]
    depth = params["depth"]
    dropout_rate = params["dropout"]
    l2_reg = params["l2"]

    for i in range(depth):
        model.add(layers.Dense(
            units // (2**i),
            activation="relu",
            kernel_regularizer=regularizers.l2(l2_reg)
        ))
        model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=params["lr"]),
        loss="mse",
        metrics=["mae"]
    )

    return model

# -----
# Zufällige Hyperparameter ziehen
# -----
def sample_params():
    return {
        "units": random.choice([32, 64, 128, 256]),
        "depth": random.choice([2, 3, 4, 5, 6]),
        "lr": 10 ** np.random.uniform(-5, -2),
        "l2": 10 ** np.random.uniform(-6, -2),
        "dropout": np.random.uniform(0.1, 0.6),
        "batch_size": random.choice([16, 32, 64]),
        "epochs": random.randint(300, 400)
    }

```

```

# -----
# Ein einzelner Run
# -----
def train_one_run(run_id, X_train, y_train, X_val, y_val, X_test, y_test):

    params = sample_params()
    model = build_model(X_train.shape[1], params)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=params["epochs"],
        batch_size=params["batch_size"],
        verbose=0,
        callbacks=[
            keras.callbacks.EarlyStopping(
                patience=7,
                restore_best_weights=True
            )
        ]
    )

    # Evaluation
    y_test_pred = model.predict(X_test, verbose=0)
    y_val_pred = model.predict(X_val, verbose=0)

    from sklearn.metrics import r2_score

    result = {
        "run_id": run_id,
        "R2 Val": r2_score(y_val, y_val_pred),
        "R2 Test": r2_score(y_test, y_test_pred),
        **params
    }

    return result, model

# -----
# Random Search (parallel)
# -----
def random_search(
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    n_runs=20,

```

```

    n_jobs=1  # >1 = parallel
):

    results = []
    models = []

    runs = Parallel(n_jobs=n_jobs)(
        delayed(train_one_run)(
            i, X_train, y_train, X_val, y_val, X_test, y_test
        )
        for i in range(n_runs)
    )

    for result, model in runs:
        results.append(result)
        models.append(model)

    df = pd.DataFrame(results)

    # Bestes Modell anhand Validation auswählen!
    best_idx = df["R2 Val"].idxmax()

    best_model = models[best_idx]
    best_params = results[best_idx]

    return best_model, best_params, df

```

```

[ ]: best_model, best_params, results_df = random_search(
    X_train_engineered, y_train_min0_max1,
    X_val_engineered, y_val_min0_max1,
    X_test_engineered, y_test_min0_max1,
    n_runs=200, ##### HIER #####
    n_jobs=4  # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R2 Val", ascending=False).head()

```

```

[ ]: results_df.sort_values("R2 Test", ascending=False).head(50)

```

Ich könnt mir vorstellen, dass es ein Problem sein könnte, dass die Architektur der Netze fest vorgegeben ist. z.B. 128-64-32. Auszuprobieren wäre, was bei variabler Struktur passiert. Ich setzte einen zerfallsfaktor ein, der andere Architekturen erlaubt.

```
[ ]: import numpy as np
import pandas as pd
import random
from joblib import Parallel, delayed
from tensorflow import keras
from tensorflow.keras import layers, regularizers

# -----
# Modell bauen (abhängig von HPs)
# -----
def build_model(input_dim, params):
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))

    units = params["units"]
    depth = params["depth"]
    shrink = params["shrink_factor"]

    for i in range(depth):
        current_units = int(units / (shrink ** i))

        model.add(layers.Dense(
            current_units,
            activation="relu",
            kernel_regularizer=regularizers.l2(params["l2"])
        ))
        model.add(layers.Dropout(params["dropout"]))

    model.add(layers.Dense(1))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=params["lr"]),
        loss="mse",
        metrics=["mae"]
    )

    return model

# -----
# Zufällige Hyperparameter ziehen
# -----
def sample_params():
    return {
        "units": random.choice([64, 128, 256]),
        "depth": random.choice([2, 3, 4]),
    }
```

```

        "shrink_factor": random.choice([1.0, 1.5, 2.0]), # NEU
        "lr": 10 ** np.random.uniform(-4, -2),
        "l2": 10 ** np.random.uniform(-5, -3),
        "dropout": np.random.uniform(0.1, 0.3),
        "batch_size": random.choice([16, 32]),
        "epochs": random.randint(200, 300)
    }

# -----
# Ein einzelner Run
# -----
def train_one_run(run_id, X_train, y_train, X_val, y_val, X_test, y_test):

    params = sample_params()
    model = build_model(X_train.shape[1], params)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=params["epochs"],
        batch_size=params["batch_size"],
        verbose=0,
        callbacks=[
            keras.callbacks.EarlyStopping(
                patience=7,
                restore_best_weights=True
            )
        ]
    )

    # Evaluation
    y_test_pred = model.predict(X_test, verbose=0)
    y_val_pred = model.predict(X_val, verbose=0)

    from sklearn.metrics import r2_score

    result = {
        "run_id": run_id,
        "R2 Val": r2_score(y_val, y_val_pred),
        "R2 Test": r2_score(y_test, y_test_pred),
        **params
    }

    return result, model

```

```

# -----
# Random Search (parallel)
# -----
def random_search(
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    n_runs=20,
    n_jobs=1  # >1 = parallel
):

    results = []
    models = []

    runs = Parallel(n_jobs=n_jobs)(
        delayed(train_one_run)(
            i, X_train, y_train, X_val, y_val, X_test, y_test
        )
        for i in range(n_runs)
    )

    for result, model in runs:
        results.append(result)
        models.append(model)

    df = pd.DataFrame(results)

    # Bestes Modell anhand Validation auswählen!
    best_idx = df["R2 Val"].idxmax()

    best_model = models[best_idx]
    best_params = results[best_idx]

    return best_model, best_params, df

```

```

[ ]: best_model, best_params, results_df = random_search(
    X_train_engineered, y_train_min0_max1,
    X_val_engineered, y_val_min0_max1,
    X_test_engineered, y_test_min0_max1,
    n_runs=200, ##### HIER #####
    n_jobs=4  # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R2 Val", ascending=False).head()

```

```
[ ]: results_df.to_csv("ensemble_models_2.csv", index=False)
```

ich möchte nochmal neue Features ausprobieren, da ich glaube, dass Feature Engineering mehr Potential hat, als ich bisher herausholen konnte. - "MedInc" * "AveRooms" - "MedInc" * "Population" - Distanz zum Zentrum: $((\text{"Latitude"} - 36)^2 + (\text{"Longitude"} + 119)^2)^{-1/2}$ - "AveOcc" 2 - $\log(\text{"Population"})$

```
[ ]: def better_feature_engineering(df, feature_names):
    df = pd.DataFrame(df, columns=feature_names).copy()
    eps = 1e-8 # Vermeidung von Division durch Null
    df["Inc_Rooms"] = df["MedInc"] * df["AveRooms"]
    df["Inc_Pop"] = df["MedInc"] * df["Population"]

    df["distance_to_center"] = np.sqrt((df["Latitude"] - 36)**2 +
    ↪(df["Longitude"] + 119)**2)
    df["sq_Occup"] = df["AveOccup"] ** 2
    df["log_population"] = np.log1p(df["Population"])
    return df.values
```

```
[ ]: # Feature Engineering auf minmax-skalierten Daten
feature_names = ["MedInc", "HouseAge", "AveRooms", "AveBedrms",
                 "Population", "AveOccup", "Latitude", "Longitude"]

X_train_engineered = better_feature_engineering(X_train_min0_max1,
    ↪feature_names)
X_val_engineered    = better_feature_engineering(X_val_min0_max1, feature_names)
X_test_engineered   = better_feature_engineering(X_test_min0_max1, feature_names)

print("Train shape:", X_train_engineered.shape)
print("Val shape:",   X_val_engineered.shape)
print("Test shape:",  X_test_engineered.shape)
```

```
[ ]: best_model, best_params, results_df = random_search(
    X_train_engineered, y_train_min0_max1,
    X_val_engineered, y_val_min0_max1,
    X_test_engineered, y_test_min0_max1,
    n_runs=200, ##### HIER #####
    n_jobs=4 # parallel auf 4 Kernen
)

print("\nBest Hyperparameters:")
print(best_params)

results_df.sort_values("R² Val", ascending=False).head()
```

```
[ ]:
```